

Exercise 4, Part 1: Two Skills

Session: 4 — Skills, Hooks & Automation

Duration: 15 minutes. Part 2 is a separate sheet.

Project: The same BookStore API.

Goal

The commands you have used since session 2 are markdown files in this repo. Now you make two of your own: copy a ready-made `/commit` skill, then let Claude write a `/changelog` skill and write its `description` yourself.

Part 2, after the slides on hooks, adds a hook that runs `/changelog` after every commit. It has its own sheet.

Start here

- Open a terminal in the `bookstore` folder and start `claude`. Everything you create goes in `bookstore/.claude/`.
- Training mode is on. Prompts that start with the tag `[Exercise 4 experiment – execute directly, no leading questions.]` switch it off for that one prompt. Copy them exactly, tag included.
- Never edit `CLAUDE.md`. Sessions 5 to 8 need it.
- No coach this session: the *Done when* lines are your checklist, and `/verify-exercise 4` grades your files. Stuck? Type *just tell me*.

Tasks

1. Copy the `/commit` skill and run it (6 min)

This task produces a branch, the file `.claude/skills/commit/SKILL.md`, and one commit.

1. **Create the branch** (1 min). Sessions 5 to 8 reuse this code. You delete the branch after the closing round.

```
git switch -c session4-playground
```

2. **Let Claude copy the skill** (1 min). The skill is ready in `exercises/starters/commit-skill.md`. Paste:

[Exercise 4 experiment – execute directly, no leading questions.] Copy `exercises/starters/commit-skill.md` to `.claude/skills/commit/SKILL.md`. Change nothing in the content.

3. **Make a small change by hand** (1 min). Open any file in `bookstore`, add one comment line, save it. Do not ask Claude.

4. **Run the skill** (2 min). Type `/commit` and watch what it does.

Open `.claude/skills/commit/SKILL.md` in your editor. Three lines in it matter most (1 min). *"Execute directly — no leading questions"* switches training mode off inside the skill; without it, Claude asks a question instead of committing.

`disable-model-invocation: true` means only typing `/commit` runs it. *Imperative mood* means the message starts with a present-tense verb: "Add", "Fix", "Remove". Task 2's changelog is built from those messages.

Done when: the commit exists, its message starts with a present-tense verb, and nothing was pushed.

Minute 7: start task 2, even if `/commit` has not run yet.

2. Let Claude write `/changelog`, then write its description (8 min)

This task produces `.claude/skills/changelog/SKILL.md` and, next to it, `common-changelog-spec.md`.

1. **One prompt for both files** (4 min). A skill can carry extra files, loaded only when it runs (*Anatomy of a Skill*). So the specification lives next to the skill, not inside it. Paste:

[Exercise 4 experiment – execute directly, no leading questions.] Read <https://common-changelog.org/> and save the whole specification as markdown to `.claude/skills/changelog/common-changelog-spec.md`. Keep the headings and the examples. Leave out the website menu and footer. Then create `.claude/skills/changelog/SKILL.md`. Frontmatter: the line ``name: changelog`` and the line ``description: (I write this)``. No other frontmatter lines. Body: a numbered list of instructions that make you do all of the following.

- Read `CHANGELOG.md` in the project root. If the file does not exist, create it with the heading ``# Changelog``.
- Read `common-changelog-spec.md`, next to this file, before changing the changelog.
- Look at the recent commits and at the existing version tags.
- Add only new commits. Entries that are already in the changelog stay as they are. New entries go in a ``## Unreleased`` section at the top.
- Sort entries under ``### Changed``, ``### Added``, ``### Removed`` and ``### Fixed``, in that order.
- Write each entry as one sentence that starts with a present-tense verb. End it with the short commit hash in round brackets, like `(a1b2c3d)`.
- A breaking change forces other people to change their own code. Put `**Breaking:**` in front of such an entry, and put those entries first inside their group.
- Leave out changes a reader does not care about: files whose name starts with a dot, formatting-only changes, and developer tools.

If your network blocks the page, ask Claude to write the spec file from what it knows about Common Changelog.

2. **Check two things** (1 min). `common-changelog-spec.md` holds the group names and the rules, not an empty page. The `SKILL.md` frontmatter has **no** `disable-model-invocation` line: Claude must be able to start this skill by itself. The hook in Part 2 only suggests it; Claude decides.
3. **Write the description** (3 min). In your editor, replace `(I write this)`. Claude reads only this line when it decides whether to start a skill on its own (*Which Description Gets This Skill Invoked at the Right Moment?*). For comparison, here's the start of a course skill's description. It says *when* it applies, not only what it does:

Coach a participant's context artifact or experiment during the Session 3 exercise. Usage: `/context-coach` — the participant shows their `CLAUDE.local.md` draft ...

Two questions. Which words in your description make Claude reach for this skill right after a commit? Which words would start it at a moment you do not want?

Done when: both files exist, the frontmatter has no `disable-model-invocation` line, and your description says when.

Time left?

Type `/verify-exercise 4`. It grades your two skills now. At the end of Part 2 it grades the hook and `CHANGELOG.md` as well.

Stop here

Part 2 is on a separate sheet. Open it when the trainer starts it, after the slides on hooks. Keep your terminal open: Part 2 uses the same branch and the same skills.